

What is CTF?

An IT security puzzle

Topics

- Computer security
- Computer science
- Networking
- IT operation

Objective: Find a way to get the flag in a limited time

Flag



Hidden or seems impossible to access through normal ways

Types:

- A file
- A message
- A series of characters

Solutions

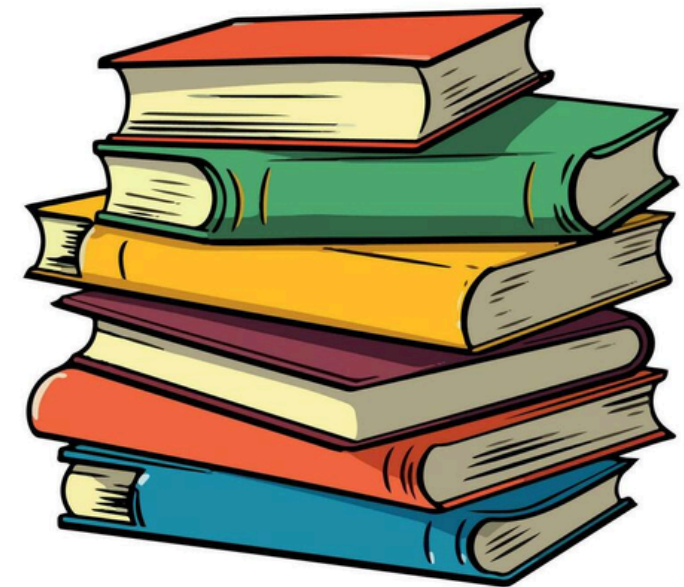
- Writing a code snippet
- Exploiting a known vulnerabilities (or maybe zero-day)
- Script (e.g. bash)
- Using a tool (e.g. debugger, static code analysers, proxies)
- Finding an algorithm to get the flag • Engineer a way to get to the flag

Why CTF?

We never learn to drive by reading a book!

We like games!

- We enjoy learning through games.
- We learn skillset as well as the theory.
- In a game we need to put the theory in practice.



Why CTF?

1. Understand the concept
2. Get hands-on skillset
3. Learn a new computer security techniques
4. Improve problem solving skills
5. Improves the thought process to think like a hacker/attacker
6. Helps to think out of the box and intuitively

warmup

```
f = int(input("flag:"))
```

```
if f - 500 > 400 and f + 100 < 1002:
```

```
    print("success")
```

```
else: print("failure")
```

warmup

```
f = int(input("flag:"))  
if f - 500 > 400 and f + 100 < 1002:  
    print("success")  
else: print("failure")
```

The only value of flag that satisfies this condition is 901

warmup

```
f = input("Enter password: ")
```

```
a = int(f[0])
```

```
b = int(f[1])
```

```
c = int(f[2])
```

```
d = int(f[3])
```

```
if b != c:
```

```
    print('ACCESS DENIED')
```

```
elif c != d:
```

```
    print('ACCESS DENIED')
```

```
elif a != b - 2:
```

```
    print('ACCESS DENIED')
```

```
elif a + b + c == 25:
```

```
    print('ACCESS GRANTED')
```

```
else:
```

```
    print('ACCESS DENIED')
```


warmup

```
f = input("Enter password: ")
```

```
a = int(f[0])
```

```
b = int(f[1])
```

```
c = int(f[2])
```

```
d = int(f[3])
```

a, b, c, d are the 4 digits of the password

```
if b != c:
```

```
    print('ACCESS DENIED')
```

```
elif c != d:
```

```
    print('ACCESS DENIED')
```

```
elif a != b - 2:
```

```
    print('ACCESS DENIED')
```

```
elif a + b + c == 25:
```

```
    print('ACCESS GRANTED')
```

```
else:
```

```
    print('ACCESS DENIED')
```

a must equal b-2

If b isn't equal to c, or if c isn't equal to d, it denies us.

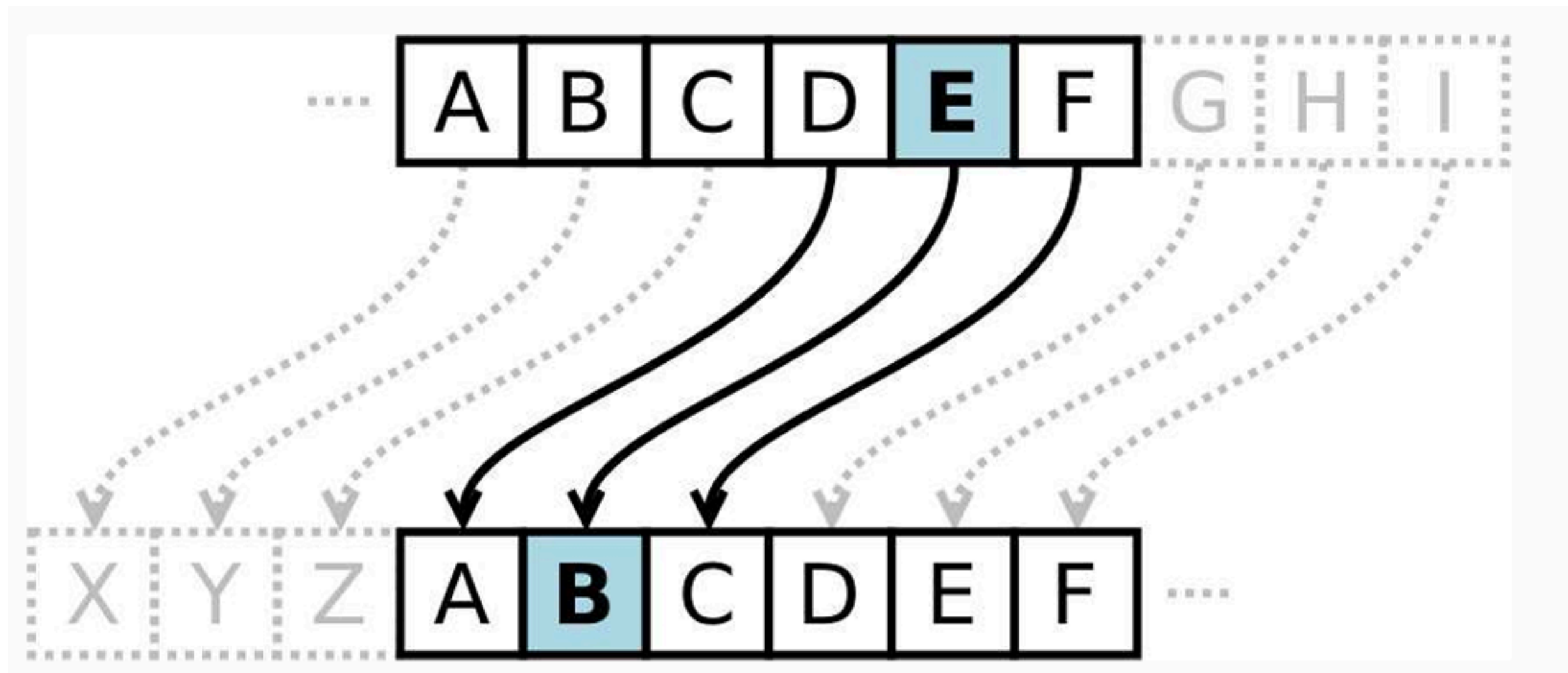
So b, c, and d must be equal.

a+b+c must equal 25 time for some math...

What is the secret message?

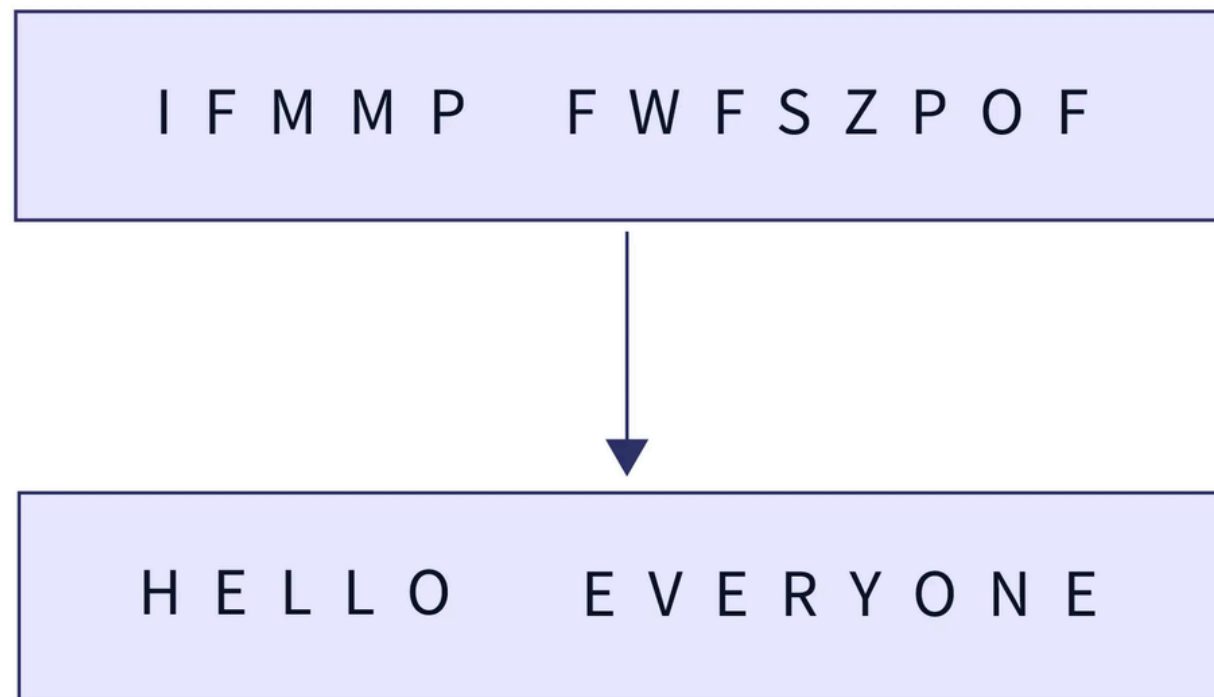
I F M M P F W F S Z P O F

The Caesar cipher is one of the simplest encryption algorithms in which every latin letter of a given string is simply shifted cyclically by a certain offset.



What is the secret message?

I F M M P F W F S Z P O F



Core Categories

Almost every challenge you will ever see falls into one of these five buckets. Recognising the bucket is half the solve.

WEB

Web Exploitation

Attack the application logic of a site — inputs, cookies, hidden endpoints.

`DevTools, Burp, curl`

CRYPTO

Cryptography

Break weak ciphers or misused modern ones using mathematical structure.

`CyberChef, Python`

FOR

Forensics

Recover data that was hidden, deleted or captured in transit.

`Wireshark, exiftool`

RE

Reverse Engineering

Read a compiled binary backwards to recover the logic inside it.

`Ghidra, IDA Pro`

PWN

Binary Exploitation

Corrupt a running program's memory to hijack its execution.

`gdb, pwntools`

Web Exploitation

CLIENT SIDE vs SERVER SIDE

Anything shipped to your browser is yours to inspect and modify: HTML, CSS, JavaScript, cookies, local storage, hidden form fields.

CLIENT SIDE vs SERVER SIDE

Server-side checks are the only real checks. When a developer validates only in JavaScript, the control is decoration and that is the beginner web category.

You can only exploit what you can see and use.

Cookies

Input Fields

DevTools

Network Requests

Logs

Source Code

Cryptography

CLASSICAL CIPHERS

Caesar, Vigenère, Atbash, substitution

- Tiny keyspace — brute force all 25 shifts
- Letter frequency gives it away
- Solvable by hand or one CyberChef op

$$C = (P + k) \bmod 26$$

MODERN SYMMETRIC

AES — one shared secret key

- Strong algorithm, weak usage
- ECB mode leaks repeating patterns
- Reused IV or hardcoded key = broken

AES-128 / AES-256, CBC · ECB · GCM

MODERN ASYMMETRIC

RSA — public and private key pair

- Security rests on factoring $n = p \times q$
- Small e or small n factors instantly
- Shared prime across keys is fatal

$$c = m^e \bmod n$$

Break by analysis or using tools like cyberchef

Digital Forensics & Hidden Data

01

Network Forensics

Wireshark, tshark, tcpdump

02

Steganography

steghide, zsteg, stegsolve, Audacity

03

File Systems & Metadata

file, strings, exiftool, binwalk

Data that looks deleted, hidden or invisible is usually there just not where you were looking.



Viewable Message



Hidden Message

Reverse Engineering & Binary Exploitation

REVERSE ENGINEERING

Read the program backwards

Ghidra · IDA Pro · radare2 · objdump

PWN — BINARY EXPLOITATION

Corrupt memory to take control

gdb + pwndbg · pwntools · checksec · ROPgadget



Cheat Engine & Game Cracks

What you need

01

Browser DevTools

02

CyberChef

03

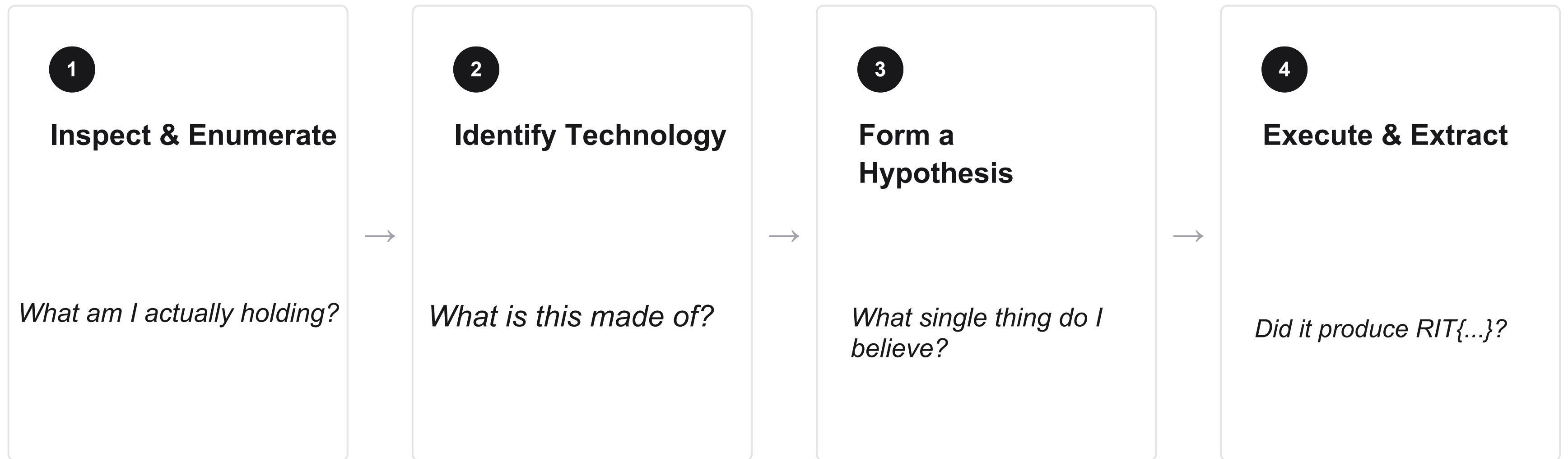
Wireshark

04

Linux CLI

PRACTICE!!

Method



TRIMM.LINK/COAT

GIST.GITHUB.COM/ANANDPRAFULL

Challenge 1:

Fragment of a Divided Soul

A fragment lies hidden beneath layers of illusion. Only those who look beyond what is visible can uncover the truth — but even then, the final form requires transformation.

<https://fragements-of-the-divided-soul.vercel.app/>

Step 1 — Open the webpage

- Load the challenge page in your browser.

Step 2 — Read the hint

- The page suggests not everything is visible.

Step 3 — Inspect with DevTools

- Right-click → Inspect → open Developer Tools.

Step 4 — Check the JavaScript file

- Look into script.js.
- You find:
- `let a = [103, 110, 105, 114];`

Step 5 — Convert ASCII values

- 103 → g
- 110 → n
- 105 → i
- 114 → r
- Result: gnir

Step 6 — Reverse the string

- gnir → ring

Step 7 — Convert “ring” back into ASCII values

- r → 114
- i → 105
- n → 110
- g → 103

Final Flag:CTF{114_105_110_103}

Challenge 2:

Chamber of Secrets

Not everything at Hogwarts is visible at first glance. Follow the hidden trail, decode the magic, and discover what lies inside the vault

<https://chamber-of-secrets-nu.vercel.app/>

1. Open the main page
 - Visit index.html
 - Page shows a Hogwarts-themed message
2. Inspect the page source
 - Right click → View Page Source
 - Hidden comment reveals path: /diagon
3. Navigate to the hidden path
 - Open diagon/index.html
4. Decode the message
 - Page contains Base64 string: U2VjcmV0X3BhdGg6lC9ncmluZ290dHM=
 - Decode → Secret_path: /gringotts
5. Navigate to /gringotts
 - Open gringotts/index.html
6. Inspect page source again
 - Hidden hint: "The key is reversed: drowssap"
7. Interpret the clue
 - Reverse string → "password"
 - Indicates something hidden/protected
8. Locate the hidden file
 - Explore directory → find flag.txt
9. Decode the final flag
 - flag.txt contains Base64: ZmxhZ3tnb2xkZW5fc25pdGNoX3NIY3JldH0=
 - Decode → flag{golden_snitch_secret}

✅ Final Flag: **flag{golden_snitch_secret}**

Challenge 3:

Colours of Hogwarts

A wizard hid a flag somewhere only a true hogwarts student would find

<https://the-colours-of-hogwarts.vercel.app>

1. Inspect the Source Code

- The page displays 38 Hogwarts-themed colour rows.
- Many fake flags are hidden in the text.

2. Identify the Anomaly

- The row labeled “Elder Wand Purple” (r30) is different.
- Instead of readable text, it contains a hexadecimal string:
- 7269744354467b70686f656e69785f6f726465725f72697365737d

3. Decode the Hex

- Convert the hex string to ASCII.

Final Flag: **ritCTF{phoenix_order_rises}**

<https://the-journey-to-riddles-diary.vercel.app/schedule.html>

The Hogwarts Express is departing and you have been chosen to board. Somewhere along this enchanted journey lies Tom Riddle's Diary, the second Horcrux, waiting to be found and destroyed. The path is hidden in plain sight, woven through tickets, timetables, and secret stations. Trust nothing at face value. Look deeper than what the naked eye can see. Only the most vigilant witch or wizard will reach the Diary. All aboard. The clock is ticking.

<https://the-journey-to-riddles-diary.vercel.app/>

Step 1 — Board the Train (travel.html)

Open travel.html. The page displays Harry's Hogwarts Express boarding pass. look at the image carefully to find the hidden meessage (/schedule).

Step 2 — Read the Timetable (schedule.html)

Open schedule.html. You see 8 train departure times. There is a hidden HTML comment: "The minutes reveal the destination."

Extract only the minutes from each departure time:

Destination	Time	Minutes
Hogsmeade Village	09:20	20 → T
Diagon Alley	10:05	05 → E
Ministry District	11:18	18 → R
Dragon Reserve	12:13	13 → M
Owl Post Station	13:09	09 → I
Forbidden Forest Halt	14:14	14 → N
Gringotts Junction	15:21	21 → U
Knight Bus Transfer	16:19	19 → S

Convert each minute to a letter using A=1, B=2 ... Z=26.
The minutes spell: T E R M I N U S

Step 3 — Unlock the Hidden Platform (terminus.html)

Open terminus.html. A password gate appears — "Only witches and wizards who know the correct platform may continue."

The answer is the most famous platform in the wizarding world:
platform93/4

Step 4 — Destroy the Horcrux (second_horcrux.html)

The page reveals Tom Riddle's Diary and provides a download link for diary.png. The instruction says: "Study it carefully. The secret it holds is your next clue."

Run ExifTool on the diary image or upload the image in aperisolve(website).The flag is embedded in the image metadata.

step 5:decode the string using Base64. **ritCTF{gaunts_ring_horcrux_shattered_by_dumbledore}**

Step 3 — Unlock the Hidden Platform (terminus.html)

Open terminus.html. A password gate appears — "Only witches and wizards who know the correct platform may continue."

The answer is the most famous platform in the wizarding world:
platform93/4

Step 4 — Destroy the Horcrux (second_horcrux.html)

The page reveals Tom Riddle's Diary and provides a download link for diary.png. The instruction says: "Study it carefully. The secret it holds is your next clue."

Run ExifTool on the diary image or upload the image in aperisolve(website). The flag is embedded in the image metadata.

step 5: decode the string using Base64.

ritCTF{gaunts_ring_horcrux_shattered_by_dumbledore}

Challenge 4:

The Mysterious Staircase

In Hogwarts, every staircase shifts but some hide something far more unusual. Can you uncover its secret?

<https://mysteriousstaircase.vercel.app/>

Step 1 — Unzip the challenge

- Download mysterious_staircase_ctf.zip
- Unzip: unzip mysterious_staircase_ctf.zip
- Move into directory: cd mysterious_staircase
- Files inside:
 - stairs_old.jpg
 - zindex.html

Step 2 — Open the webpage and read hints

- Open zindex.html in browser
- Page shows Hogwarts staircase story + clues:
 - “Old images hide secrets” → steganography
 - “Thirteen turns” → ROT13 cipher
 - Password hint: "magic"

Step 3 — Extract hidden data from image

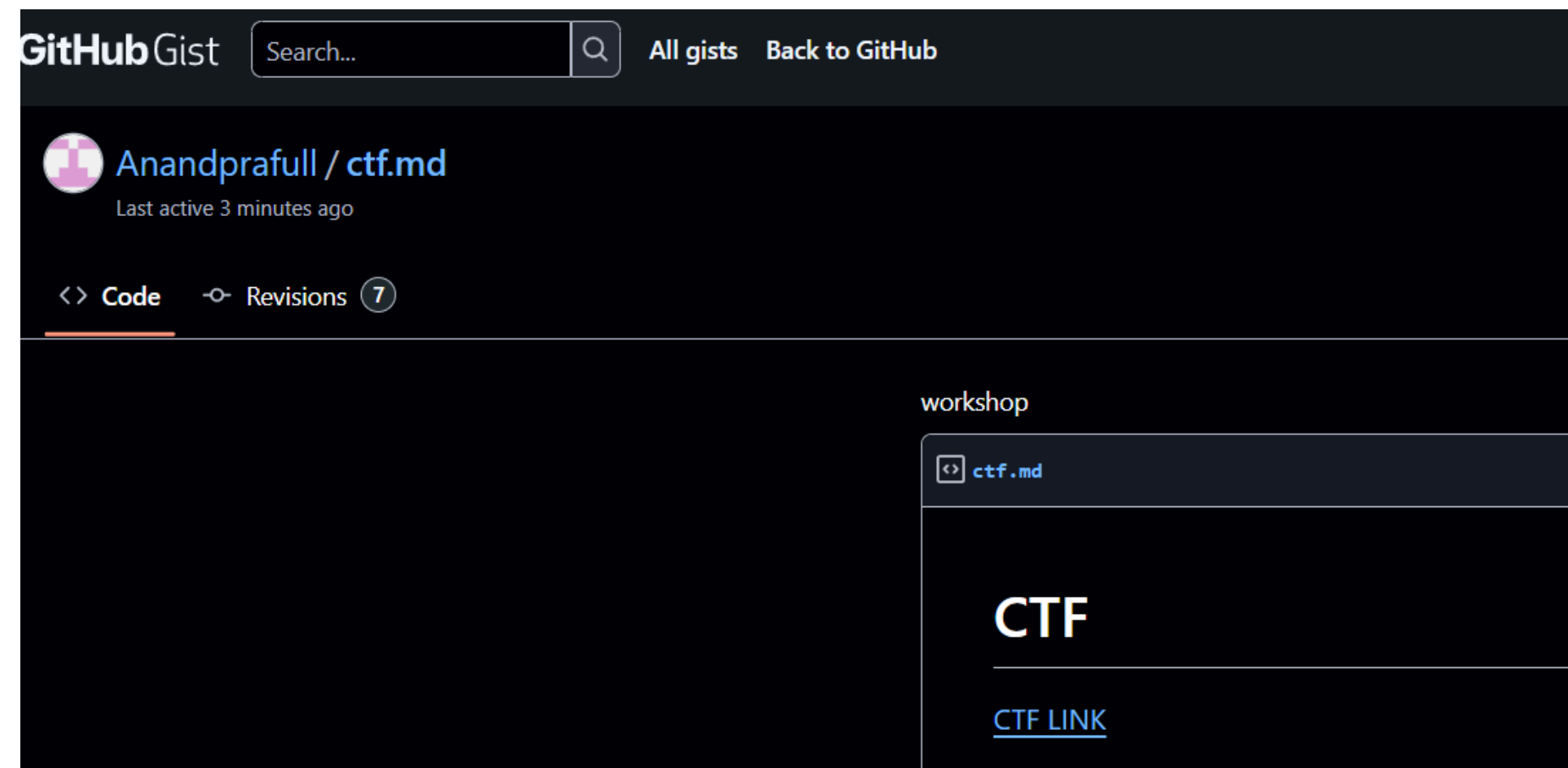
- Using Steghide (terminal):
- steghide extract -sf stairs_old.jpg
- # Enter passphrase: magic
- # Output: flag.txt
- Or use AperiSolve (upload image, enter password, download extracted file).

Step 4 — Decode the flag

- View flag.txt:
- 6576675047537b6775725f66676e7665665f79726e715f67625f7362656f76717172615f6e65726e7d
- Convert hex → ASCII:
- evgPGS{gur_fgnvef_yrnq_gb_sbeovqqra_nern}
- Apply ROT13:

Final Flag: **ritCTF{the_stairs_lead_to_forbidden_area}**

T1,123	T6,123	T15,123
T2,123	T7,123	T16,123
T3,123	T8,123	T17,123
T4,123	T9,123	T18,123
T5,123	T10,123	T19,123
	T11,123	T20,123
	T12,123	
	T13,123	
	T14,123	



<https://codes-decimal-erp-victoria.trycloudflare.com/>



Tea Break